

SOURCE CODE EXTRACT

Software Title: HealthX: A Secure Healthcare Professional Management Platform with Authentication and Profile Management System

Author: DIKSHANT SOHAL
Year of Creation: 2026

FILE: server/index.js

```
const express = require("express");
const cors = require("cors");
const dotenv = require("dotenv");
const wsExpress = require('express-ws');
const { DoctorRouter, DoctorAuthRouter } = require("../route/doctorRoute");
const { PatientRoute, PatientAuthRoute } =
require("../route/patientRoute");
const {
  verifyPatientToken,
  verifyDoctorToken,
  getTokenType,
} = require("../middleware/jwtparser");
const { connect } = require("../db/connect");
const app = express();
dotenv.config();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use(cors());
app.use("/doctor-auth", DoctorAuthRouter); // login signup etc
app.use("/doctor-appointment", verifyDoctorToken, DoctorRouter);
app.use("/patient-auth", PatientAuthRoute); // login signup etc
app.use("/patient-appointment", verifyPatientToken, PatientRoute);
wsExpress(app)
const { wsRouter, restRouter } = require("../route/chatRoute");
app.use("/get-appointment-chats", restRouter)
app.use('/ws-chat', wsRouter)
app.get('/get-token-type', getTokenType)
const start = async () => {
  try {
    const db_url = process.env.MONGODB_URI;
    const port = process.env.PORT || 8080;
    connect(db_url);
    app.listen(port, () => {
      console.log(`server up n running on ${port} and connected to
db....`)
    });
  } catch (error) {
    console.log(error);
    console.log("server did not started because of error in db
connection...");
  }
}
```

```
    }  
  };  
  start();
```

```
-----  
FILE: server/route/patientroute.js  
-----
```

```
const  
{loginUser,signupUser,forgotPassword,verifyEmailToken,verifyForgotPassword  
Token} = require('../controller/patient/PatientAuthenticationHandler')  
const {getPatientAppointments,acceptYourDoctorForTreatment,reviewDoctor,  
createAppointment,getuserprofile,pastappointment} =  
require('../controller/patient/PatientAppointmentHandler')  
const PatientRoute = require('express').Router() // jwt require  
PatientRoute.post('/create-appointment',createAppointment )  
PatientRoute.route('/get-patient-  
appointments').get(getPatientAppointments)  
PatientRoute.route('/accept-your-  
doctor').post(acceptYourDoctorForTreatment);  
PatientRoute.route('/review-doctor').post(reviewDoctor)  
PatientRoute.route('/past-appointments').get(pastappointment)  
PatientRoute.route('/user-profile').get(getuserprofile);
```

```
const PatientAuthRoute = require('express').Router() // not jwt
```

```
PatientAuthRoute.route('/login').post(loginUser)  
PatientAuthRoute.route('/signup').post(signupUser)  
PatientAuthRoute.route('/reset-password').post(forgotPassword)  
PatientAuthRoute.route('/verify-email-token').post(verifyEmailToken)  
PatientAuthRoute.route('/verify-forgot-password-  
token').post(verifyForgotPasswordToken)  
module.exports = {PatientRoute,PatientAuthRoute}
```

```
-----  
FILE: server/route/doctorroute.js  
-----
```

```
const { getAllAppointments, getDoctorAppointments,  
addPresentDoctor,getuserprofile ,updateAppointmentProgress,getreview} =  
require('../controller/doctor/DoctorAppointmentHandler')  
const { loginUser, signupUser, forgotPassword, verifyEmailToken,  
verifyForgotPasswordToken} =  
require('../controller/doctor/DoctorAuthenticationHandler')
```

```

const DoctorRouter = require('express').Router() // jwt required to
accesss
DoctorRouter.post('/add-present-doctor', addPresentDoctor) //first
response to a patients appointment that i am available for your problem
DoctorRouter.get('/get-all-appointments', getAllAppointments)
DoctorRouter.get('/get-my-appointments', getDoctorAppointments) // on
going appointment of this doctor
DoctorRouter.route('/user-profile').get(getUserProfile);
DoctorRouter.route('/update-appointment').post(updateAppointmentProgress);
DoctorRouter.route("/get-my-reviews").get(getreview);

```

```

const DoctorAuthRouter = require('express').Router()// no jwt , signup
login etc

```

```

DoctorAuthRouter.route('/login').post(loginUser)
DoctorAuthRouter.route('/signup').post(signupUser)
DoctorAuthRouter.route('/reset-password').post(forgotPassword)
DoctorAuthRouter.route('/verify-email-token').post(verifyEmailToken)
DoctorAuthRouter.route('/verify-forgot-password-
token').post(verifyForgotPasswordToken)

```

```

module.exports = {DoctorRouter,DoctorAuthRouter}

```

```

-----
FILE: server/route/chatroute.js
-----

```

```

const express = require("express");
const { Chat } = require("../model/chatModel");
const wsRouter = express.Router();
wsRouter.ws("/:appointmentId", (ws, req) => {
  console.log(req.params.appointmentId);
  ws.on("message", (k) => {
    console.log(k);
    const { userType, userId="", msg } = JSON.parse(k);
    putIntoChat({ userId, userType, msg }, req.params.appointmentId);
  });
  // ws.close();
});
const putIntoChat = async ({ userType, userId, msg }, appointmentId) => {
  try {
    let chat = await Chat.findOne({ appointmentChatId: appointmentId });
    if (chat) {
      chat.chat.push({
        userType,
        time: new Date(),
        userId,

```

```

        msg,
    });
    await chat.save();
} else {
    chat = new Chat({
        appointmentChatId: appointmentId,
        chat: [
            {
                userType,
                userId,
                time: new Date(),
                msg,
            },
        ],
    });
    await chat.save();
}
console.log("new msg added into chat");
return;
} catch (error) {
    console.error("Error adding message to chat:", error);
    throw error;
}
};

```

```

const restRouter = express.Router();
restRouter.get("/:id", async (req, res) => {
    const appointmentId = req.params.id;
    try {
        const appointment = await Chat.findOne({
            appointmentChatId: appointmentId,
        });
        if (appointment) {
            res.send(appointment.chat);
        } else {
            res.status(404).send("no appointment found");
        }
    } catch (error) {}
});
module.exports = { wsRouter, restRouter };

```

```

-----
FILE: server/Controller/DoctorAppointmentHandler.js
-----

```

```

const { AppointmentSchema } = require("../../model/appointmentModel");
const { DoctorSchema } = require("../../model/doctorModel");
const User = DoctorSchema;
const getAllAppointments = async (req, res) => {
    try {

```

```

const data = await AppointmentSchema.find({
  progress: "toaccept",
}).populate({
  path: "patientId",
  select: "name",
});

const modifiedData = data.map((appointment) => ({
  ...appointment.toObject(),
  patientId: appointment.patientId[0] || null,
}));

res.status(200).json(modifiedData);
} catch (error) {
  console.log(error);

  res.status(500).send({ err: error });
}
};

const updateAppointmentProgress = async (req, res) => {
  try {
    const { appointmentId, done, name } = req.body;
    console.log(appointmentId);

    if (!appointmentId) {
      return res
        .status(400)
        .json({ error: "Appointment ID and progress are required." });
    }
    let updatedAppointment
    if (done === false) {
      updatedAppointment = await AppointmentSchema.findByIdAndUpdate(
        appointmentId,
        { progress: "ongoing" },
        { new: true }
      );
    }
    else {
      updatedAppointment = await AppointmentSchema.findByIdAndUpdate(
        appointmentId,
        { progress: 'done' },
        { new: true }
      )
      updatedAppointment.doctorname = name;
      console.log(updateAppointmentProgress.doctorname + "!");

      await updatedAppointment.save();
      if (!updatedAppointment) {
        return res.status(404).json({ error: "Appointment not found." });
      }
      // No further actions needed after deletion, send response
      return res.status(200).json({ message: "Appointment deleted successfully." });
    }
  }
};

```

```

    }
    if (updatedAppointment) {
      updatedAppointment.appointedDoctorId = req.userId;
      await updatedAppointment.save();
    }
    if (!updatedAppointment) {
      return res.status(404).json({ error: "Appointment not found." });
    }

    res.status(200).json(updatedAppointment);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Error updating appointment progress."
});
  }
}

const getDoctorAppointments = async (req, res) => {
  // on going appointment of this doctor
  try {
    const data = await AppointmentSchema.find({
      appointedDoctorId: req.userId,
      progress: "ongoing",
    });
    console.log(data);

    res.send(data);

  } catch (error) {
    res.status(500).send({ err: error });
  }
};

const addPresentDoctor = async (req, res) => {
  const { appointmentId } = req.body;
  const doctorId = req.userId;
  try {
    const data = await AppointmentSchema.findById(appointmentId);
    if (!data.presentDoctorIds.includes(doctorId)) {
      data.presentDoctorIds.push(doctorId);
    }
    await data.save();
    res.send(data);
  } catch (error) {
    res.status(500).send({ err: error });
  }
};

const getUserprofile = async (req, res) => {
  try {
    const Id = req.userId;
    let user = await User.findById(Id);
    console.log(user);
    delete user.password;
    delete user.verified;
    delete user.verifyToken;
  }
};

```

```

        return res.status(200).json({ user });
    } catch (err) {
        console.log(err);
    }
};
const getreview=async(req,res)=>{
    const userId = req.userId;

    try {

        const data = await User.findById(userId);
        if (!data) {
            return res.status(404).send("Doctor not found.");
        }

        const appointmentIds = data.rating.map((rev) => rev.appointmentId);

        const appointments = await AppointmentSchema.find({
            _id: { $in: appointmentIds },
        });

        const enrichedRatings = data.rating.map((rev) => {
            const appointment = appointments.find(
                (app) => app._id.toString() === rev.appointmentId
            );
            return {
                ...rev._doc,
                problem: appointment ? appointment.problem : "No problem
description",
            };
        });

        res.status(200).send(enrichedRatings);
    } catch (err) {
        console.error(err);
        res.status(500).send("Server error.");
    }
}

module.exports = {
    getAllAppointments,
    addPresentDoctor,
    getDoctorAppointments,
    getUserprofile,
    updateAppointmentProgress,
    getreview
};

```

```

-----
FILE: server/Controller/PatientAppointmentHandler.js
-----

```

```

const { AppointmentSchema } = require("../..model/appointmentModel");
const { Chat } = require("../..model/chatModel");

```

```

const { DoctorSchema } = require("../..model/doctorModel");
const { PatientSchema } = require("../..model/patientModel");
const mongoose = require('mongoose');

const User = PatientSchema;
const createAppointment = async (req, res) => {
  try {
    const { problem, time ,patientname} = req.body;
    const userId = req.userId;
    const data = await AppointmentSchema.create({
      patientId: userId,
      time: time,
      problem: problem,
      progress: "toaccept",
      reviewed: false,
      appointedDoctorId:undefined,
      presentDoctorIds:[],
      doctorname:"",
      patientname:patientname

    });
    res.send(data);
  } catch (e) {
    console.log(e);

    res.status(500).send("error while creaating appointment");
  }
};

const getPatientAppointments = async (req, res) => {
  const userId = req.userId;
  // console.log(userId);
  const objectId = new mongoose.Types.ObjectId(userId);
  console.log(objectId);

  try {
    const appointments = await AppointmentSchema.find({
      patientId: { $elemMatch: { $eq: objectId } },
      progress: { $ne: "done" }
    }).populate({
      path: 'appointedDoctorId',
      select: 'name'
    });
    const modifiedData = appointments.map(appointment => {
      const appointmentObj = appointment.toObject();
      return {
        ...appointmentObj,
        doctorName: appointmentObj.appointedDoctorId &&
appointmentObj.appointedDoctorId.length > 0
          ? appointmentObj.appointedDoctorId[0].name
          : 'No doctor assigned',
        appointedDoctorId: appointmentObj.appointedDoctorId &&
appointmentObj.appointedDoctorId.length > 0
          ? appointmentObj.appointedDoctorId[0]._id
          : null
      }
    });
  }
};

```



```

        };
    });
    console.log(modifiedData);
    res.status(200).json(modifiedData);
} catch (error) {
    console.log(error);
    res.status(500).send({ err: error });
}
};

const acceptYourDoctorForTreatment = async (req, res) => {
    try {
        const { doctorId, appointmentId } = req.body;
        const appointment = await AppointmentSchema.findById(appointmentId);
        appointment.appointedDoctorId = doctorId;
        appointment.presentDoctorIds = [];
        appointment.save();
        appointment.progress="ongoing"
        res.send(appointment);
        return;
    } catch (e) {
        res.status(500).send("Error in accepting doctor", e);
    }
};

const pastappointment=async(req,res)=>{
    try{
        const userId=req.userId;
        const appointments=await AppointmentSchema.find({
            patientId:userId,
            progress:"done"
        }).populate({
            path:'appointedDoctorId',
            select:'name'
        })
        console.log(appointments);
        res.status(200).send(appointments);
    }catch(err){
        console.log(err);
        res.status(500).json("Error")
    }
}

const reviewDoctor = async (req, res) => {
    const { appointmentId, rating,message} = req.body;
    const userId = req.userId;

    if (rating < 1 || rating > 5) {
        return res.status(400).send("Rating must be between 1 and 5.");
    }

    try {

        const appointment = await AppointmentSchema.findById(appointmentId);
        const patientname=appointment.patientname;

        if (!appointment) {

```

```

        return res.status(404).send("Appointment not found.");
    }

    if (appointment.patientId.toString() !== userId) {
        return res
            .status(403)
            .send("You are not authorized to review this appointment.");
    }

    if (appointment.reviewed) {
        return res.status(400).send("This appointment has already been
reviewed.");
    }

    const doctor = await
DoctorSchema.findById(appointment.appointedDoctorId);
    if (!doctor) {
        return res.status(404).send("Doctor not found.");
    }

    const newReview = {
        appointmentId: appointmentId,
        patientName: patientname,
        rate: rating,
        comment: message,
    };

    doctor.rating.push(newReview);
    await doctor.save();
    console.log(doctor);

    appointment.reviewed = true;
    await appointment.save();

    res.status(200).send({
        message: "Review submitted successfully.",
        doctor,
    });
} catch (error) {
    console.error(error);
    res.status(500).send("An error occurred while submitting the
review.");
}
};

const getUserprofile=async(req,res)=>{
    try{
        const Id=req.userId;
        let user=await User.findById(Id);
        console.log(user);
        delete(user.password);
        delete(user.verified);
        delete(user.verifyToken);
        return res.status(200).json({user});
    }catch(err){

```



```

        </h2>
        <div className="mt-8 grid grid-cols-1 md:grid-cols-3 gap-6">
          <FeaturesCard
            title="Chat with Doctors"
            description="Seamlessly connect with healthcare
professionals."
            icon="ðŸŒŒ"
          />
          <FeaturesCard
            title="Appointments"
            description="Schedule and manage your appointments
effortlessly."
            icon="ðŸŒŒ"
          />
          <FeaturesCard
            title="Ongoing Appointments"
            description="Track all your active appointments at one
glance."
            icon="âŒŒ"
          />
        </div>
      </section>
    </main>
    <Footer />
  </div>
);
}

```

FILE: client/app/patient/create-new-appointment/page.tsx

```

"use client"
import React, { useState } from 'react';
import { Calendar, Clock, Send } from 'lucide-react';
import { axiosFetchPatient } from '@lib/axiosConfig';

export default function CreateAppointmentForm() {
  const [problem, setProblem] = useState('');
  const [time, setTime] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError('');
    setSuccess(false);

    try {
      const token:any=localStorage.getItem("token");
      const patientname=localStorage.getItem('patientname');
      axiosFetchPatient(token).post("/create-appointment",{

```

```

        problem,
        time,
        patientname
    })).then((res)=>{
        setSuccess(true);
        setProblem("");
        setTime("");
    })
} catch (err:any) {
    setError(err.message);
} finally {
    setLoading(false);
}
};

return (
    <div className="max-w-md mx-auto mt-10 bg-white rounded-xl shadow-md
overflow-hidden md:max-w-2xl">
        <div className="md:flex">
            <div className="p-8 w-full">
                <div className="uppercase tracking-wide text-sm text-indigo-500
font-semibold mb-1">New Appointment</div>
                <h2 className="block mt-1 text-lg leading-tight font-medium
text-black">Schedule Your Visit</h2>

                {success && (
                    <div className="mt-4 bg-green-100 border-l-4 border-green-500
text-green-700 p-4" role="alert">
                        <p>Appointment created successfully!</p>
                    </div>
                )}

                {error && (
                    <div className="mt-4 bg-red-100 border-l-4 border-red-500
text-red-700 p-4" role="alert">
                        <p>{error}</p>
                    </div>
                )}

                <form onSubmit={handleSubmit} className="mt-6">
                    <div className="mb-4">
                        <label htmlFor="problem" className="block text-gray-700
text-sm font-bold mb-2">
                            Problem Description
                        </label>
                        <textarea
                            id="problem"
                            value={problem}
                            onChange={(e) => setProblem(e.target.value)}
                            className="shadow appearance-none border rounded w-full
py-2 px-3 text-gray-700 leading-tight focus:outline-none focus:shadow-
outline"
                                rows="4"
                                required

```

```

        />
    </div>
    <div className="mb-6">
        <label htmlFor="time" className="block text-gray-700 text-sm
font-bold mb-2">
            Preferred Time
        </label>
        <div className="relative">
            <div className="absolute inset-y-0 left-0 flex items-
center pl-3 pointer-events-none">
                <Clock className="h-5 w-5 text-gray-400" />
            </div>
            <input
                id="time"
                type="datetime-local"
                value={time}
                onChange={(e) => setTime(e.target.value)}
                className="shadow appearance-none border rounded w-full
py-2 pl-10 pr-3 text-gray-700 leading-tight focus:outline-none
focus:shadow-outline"
                required
            />
        </div>
    </div>
    <div className="flex items-center justify-between">
        <button
            type="submit"
            className="bg-blue-500 hover:bg-blue-700 text-white font-
bold py-2 px-4 rounded focus:outline-none focus:shadow-outline flex items-
center"
            disabled={loading}
        >
            {loading ? 'Creating...' : (
                <>
                    <Send className="h-4 w-4 mr-2" />
                    Create Appointment
                </>
            )}
        </button>
    </div>
</form>
</div>
</div>
</div>
);
}

```